



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Linguagem de Descrição de Hardware – Unidade 1

Dr. Ney Laert Vilar Calazans

Última alteração: 10/10/2025

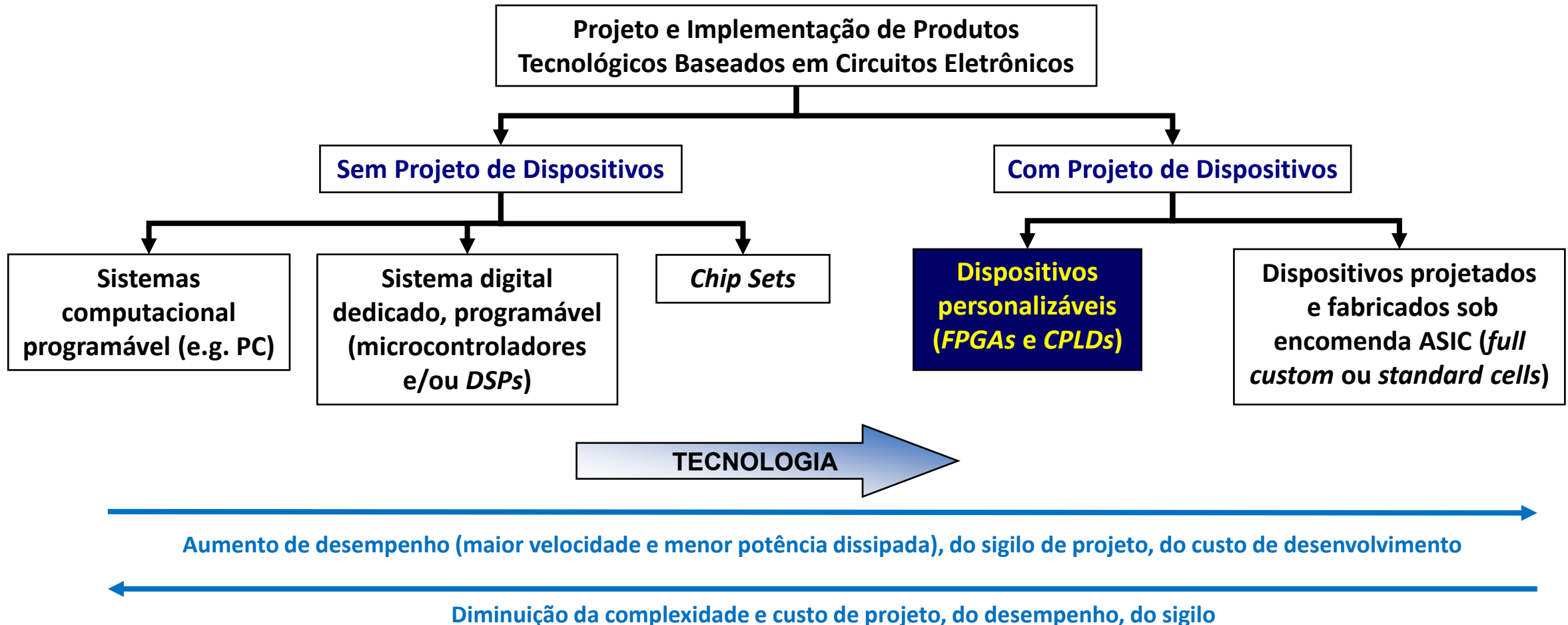
❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

INTRODUÇÃO À TECNOLOGIA DE FPGAS E À PROTOTIPAÇÃO DE HARDWARE

TEORIA, HISTÓRICO E ESTRUTURA DE FPGAs

O QUE SÃO FPGAs?

- FPGAs permitem implementar sistemas digitais diretamente de HDLs, sem os custos de fabricação de chips!

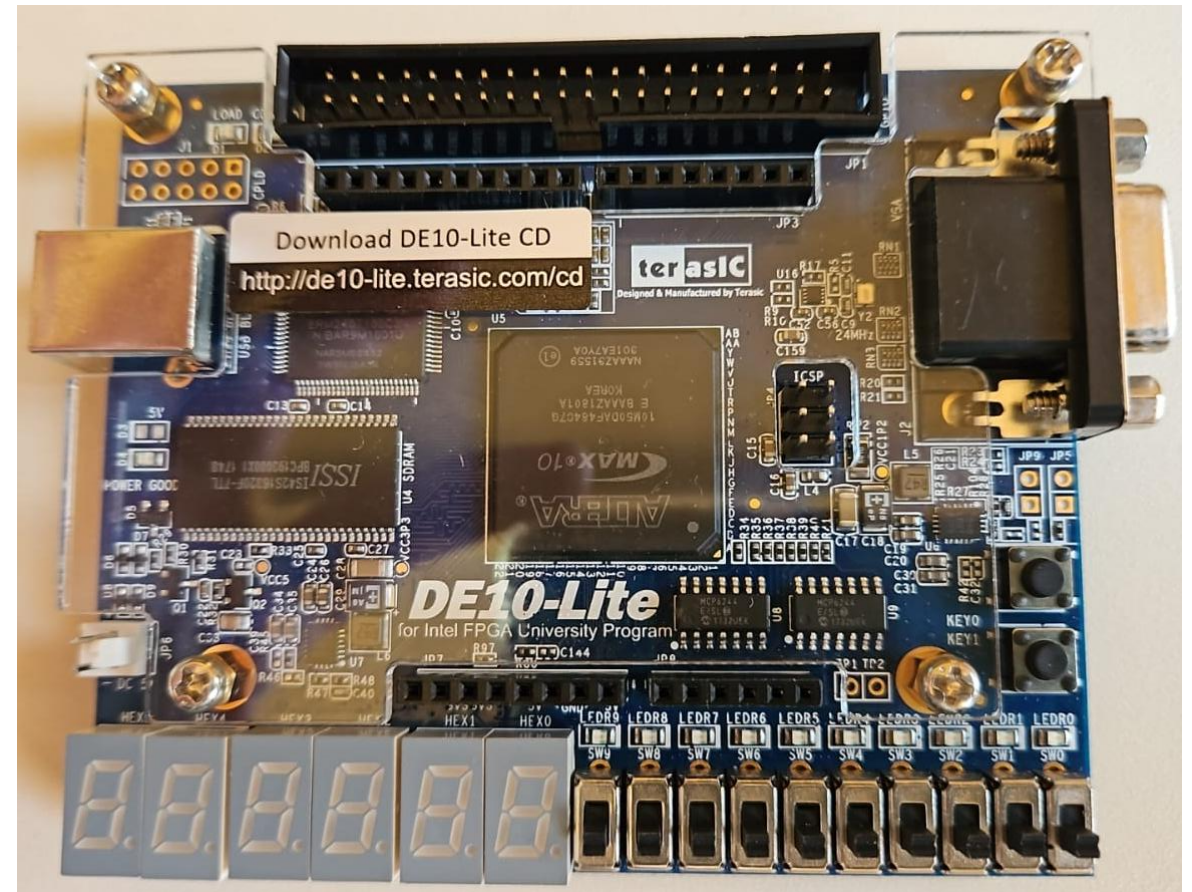


DOIS EXEMPLOS DE PLATAFORMAS BASEADAS EM FPGAS

Digilent Nexys 2



Terasic DE10-Lite

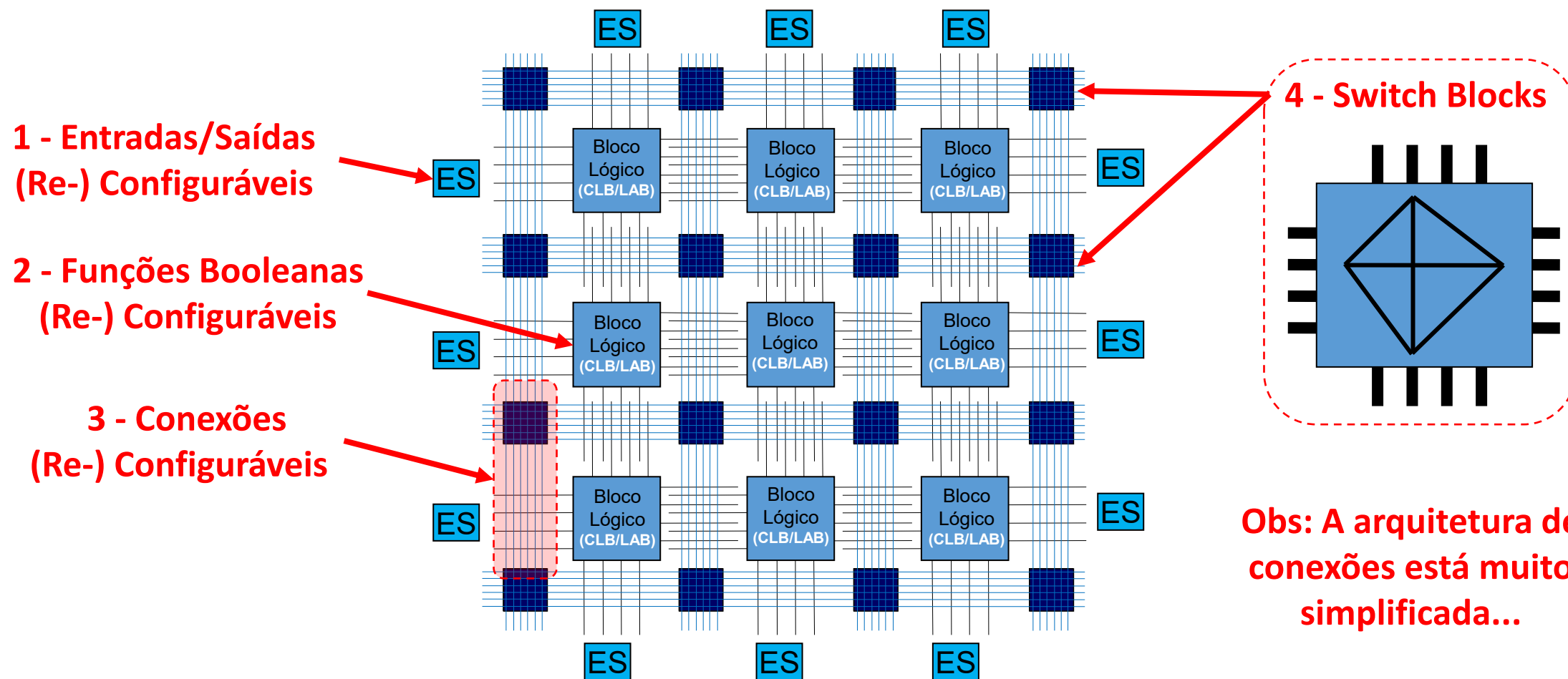


UM SLIDE SOBRE A HISTÓRIA DE FPGAS

- Primeiro, vieram PROMs (*Programmable Read Only Memories*) e *PLDs* (*Programmable Logic Devices*), matrizes de portas lógicas (re-) configuráveis
- Algumas patentes de coisas parecidas com *FPGAs* (*Field Programmable Gate Arrays*) surgiram no final dos anos 80 e início dos anos 90 (Casselman, Page, Peterson)
- Os fundadores da [Xilinx](#), Ross Freeman e Bernard Vonderschmitt, inventaram o primeiro FPGA comercial em 1985 – o XC2064
- O XC2064 tinha 64 blocos lógicos configuráveis e interconexões configuráveis entre os blocos lógicos
- O XC2064 só tinha blocos lógicos configuráveis (CLBs), cada um com duas LUTs de 3 entradas

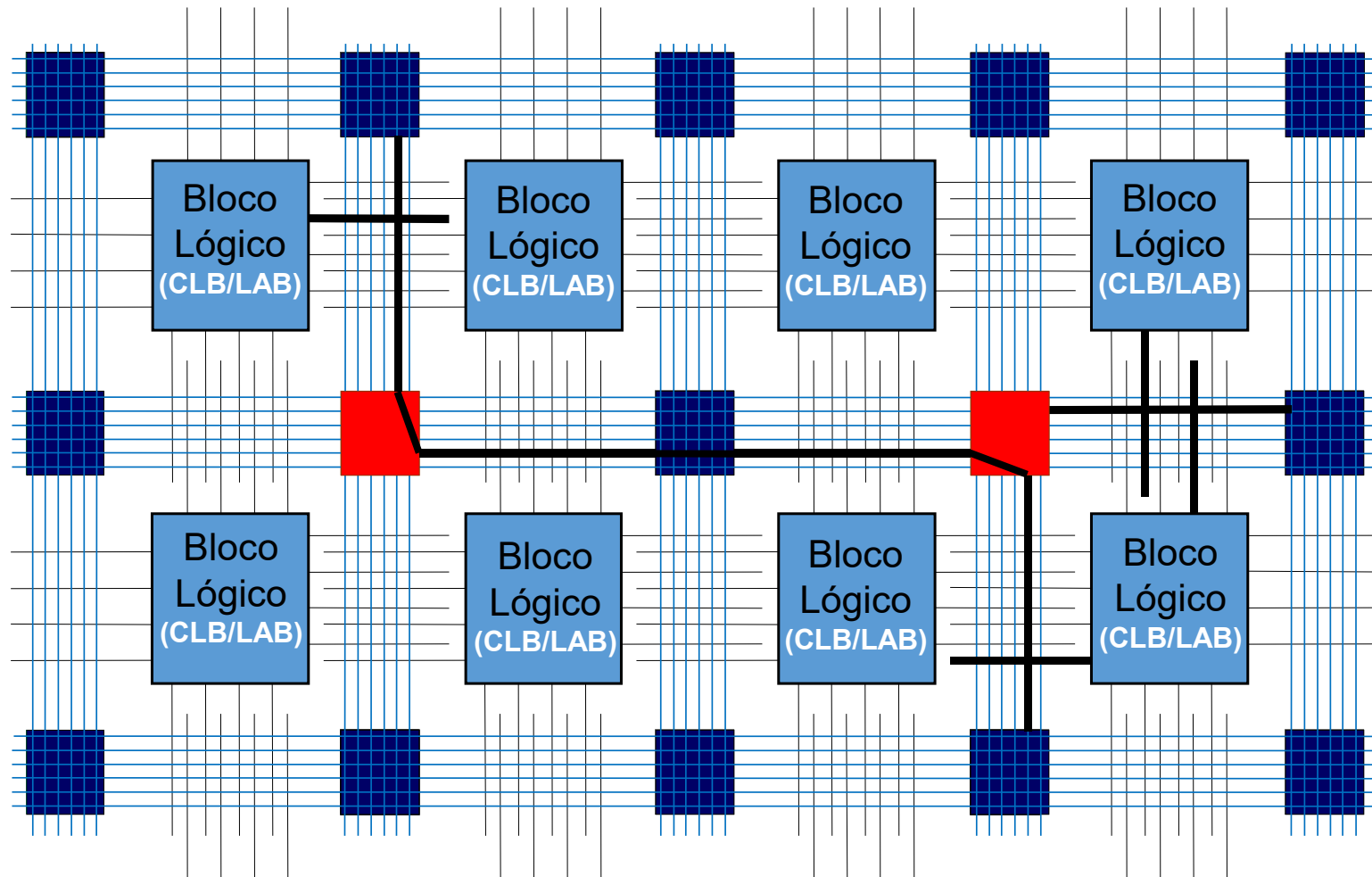
FPGAs – CONCEITOS BÁSICOS

- Matriz de *configurable logic blocks* (CLBs, nomenclatura [AMD/Xilinx](#)) ou *logic array blocks* (LABs, nomenclatura [Intel/Altera](#)), interconectados por matrizes de chaveamento



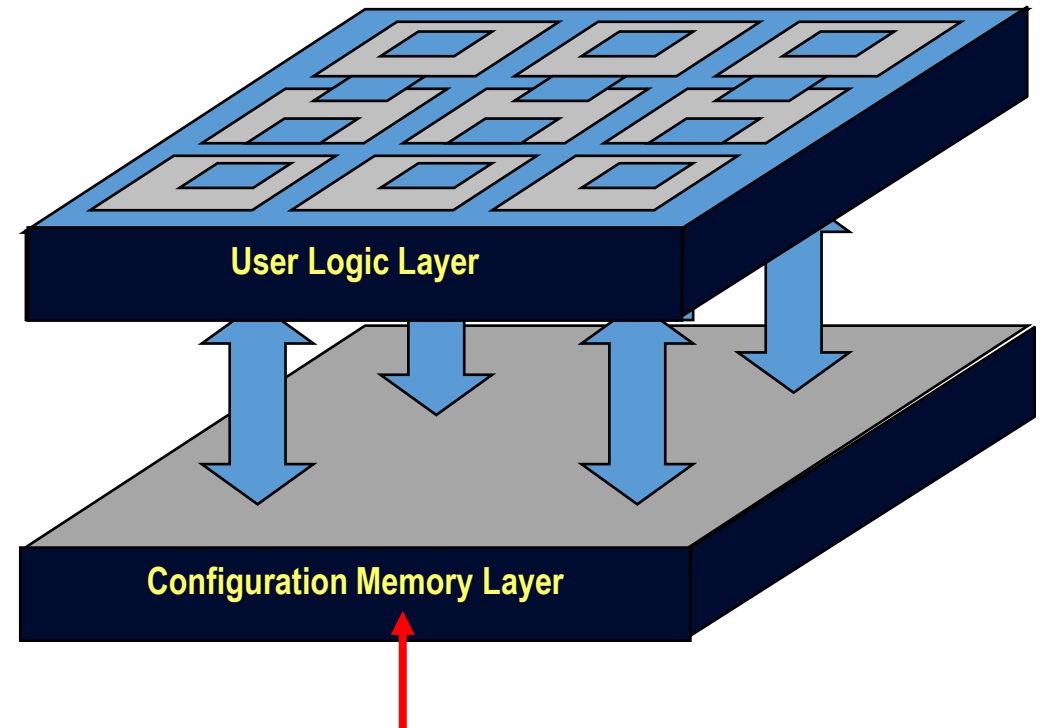
CONEXÕES ENTRE CLBS ou LABS

➤ Exemplo de conexão entre duas redes



FPGAs – CONFIGURAÇÃO (RAM-BASED)

- Um FPGA deve ser visto como um dispositivo de “duas camadas”
 1. Bloco 1 - Uma memória de configuração (90% do Hw do chip) associada a toda uma lógica que define o Bloco 2
 2. Bloco 2 - a lógica do usuário (10% do Hw)
- A memória de configuração define
 - ✓ Toda a fiação da lógica do usuário
 - ✓ As funções lógicas das LUTs (a ver...)
 - ✓ Interface externas e internas
 - ✓ Estrutura e conteúdo das memórias (ROM/RAM) do usuário
 - ✓ Configuração dos pinos de E/S



Por exemplo, na família Virtex 4, a memória de configuração ocupa algo entre 1 MByte – 4 MBytes

TECNOLOGIAS DE CONFIGURAÇÃO

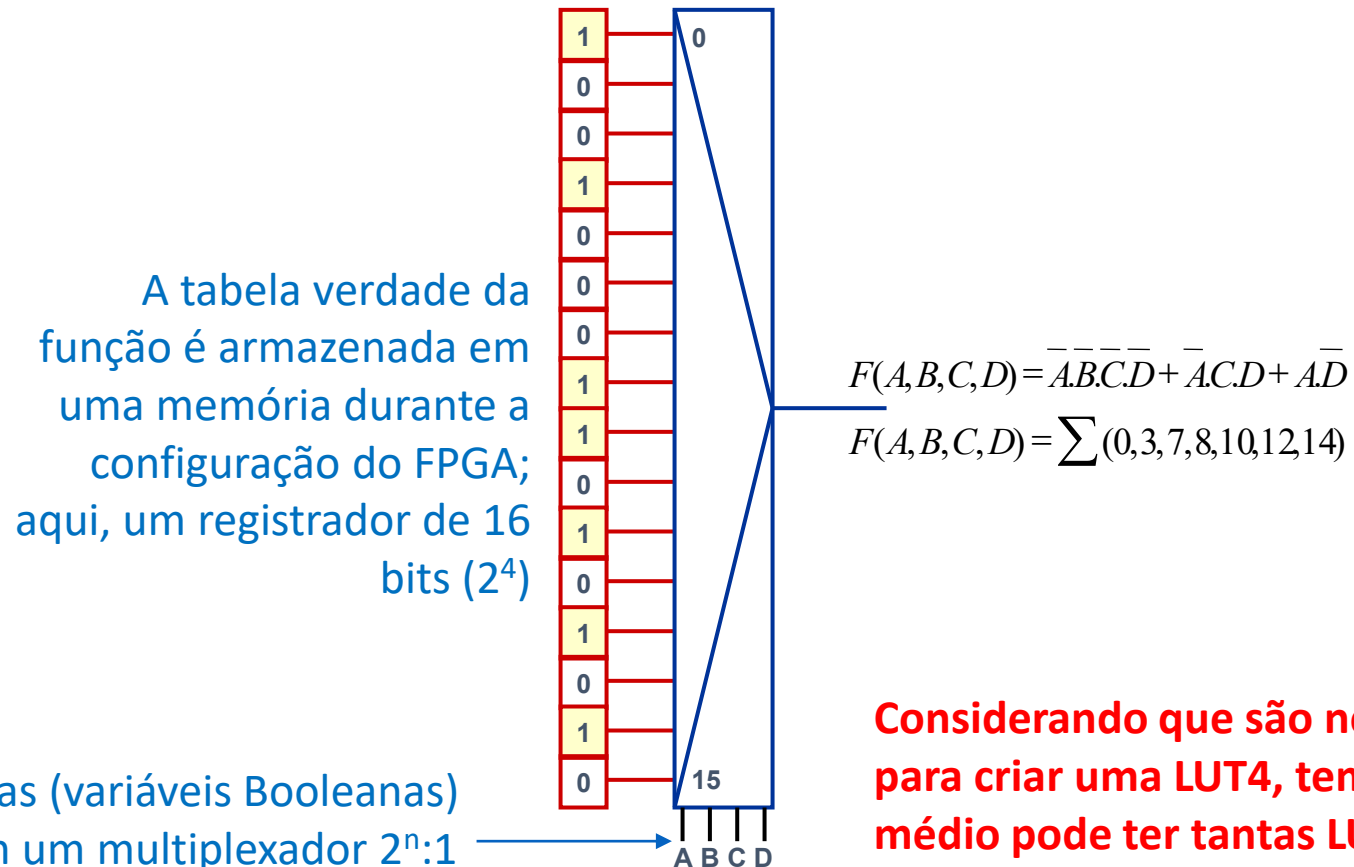
- Algumas das diferentes tecnologias usadas para definir o comportamento de um FPGA
 - ✓ **Antifusível**: Configuração uma única vez, bem barato, mas depois de configurado, fica assim “Para todo o Sempre, Amém!”
 - ✓ **(E)EPROM**: Configuração um número limitado de vezes (100 a 1000 vezes), mantida mesmo com o *chip* desconectado da alimentação
 - ✓ **Flash**: Tecnologia mais avançada que (E)EPROM, de mesma funcionalidade, mas mais durável e mais simples de configurar – **Placa DE10-Lite usa MAX10, flash-based!**
 - ✓ **SRAM**: Configuração deve ser realizada cada vez que o FPGA for alimentado, seja de forma manual, seja de forma automática (desligou, sumiu o Hw...) – **Placas Digilent Nexys e Nexys2 usam FPGAs baseados em SRAM (das famílias Spartan3 e Spartan3E, resp.)**

LUT – UM GERADOR UNIVERSAL DE FUNÇÕES

- **LUT** (*Look-Up Table*) – Um exemplo de bloco com função Booleana reconfigurável – **Implementador Universal de Funções!!!**
 - ✓ Uma porção de hardware configurável/reconfigurável capaz de implementar **qualquer** tabela verdade de n entradas
 - ✓ Para $n=4$, existem 65.536 funções implementáveis distintas (2^{2^4}) – Mas por quê?
 - Altamente flexível
 - Método mais utilizado (Xilinx [AMD], Altera [Intel] e outros fabricantes usam LUTs)

LUT – UM GERADOR UNIVERSAL DE FUNÇÕES

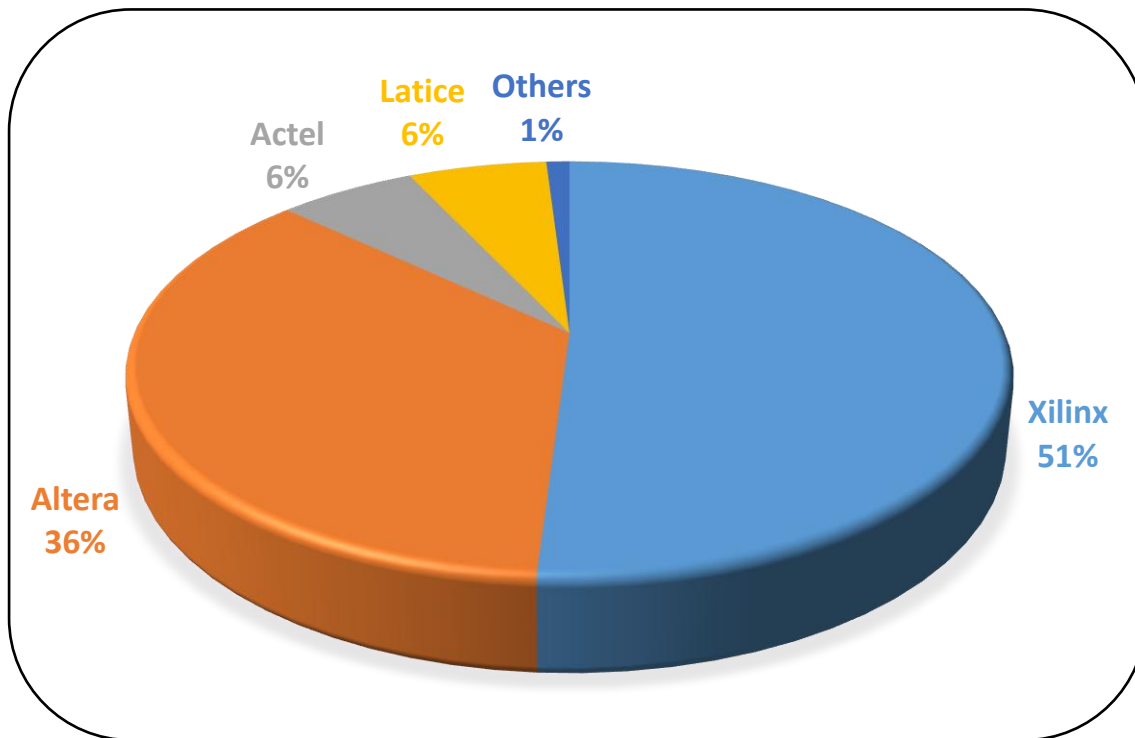
➤ Implementação física de uma LUT4 → notem como é simples!!!



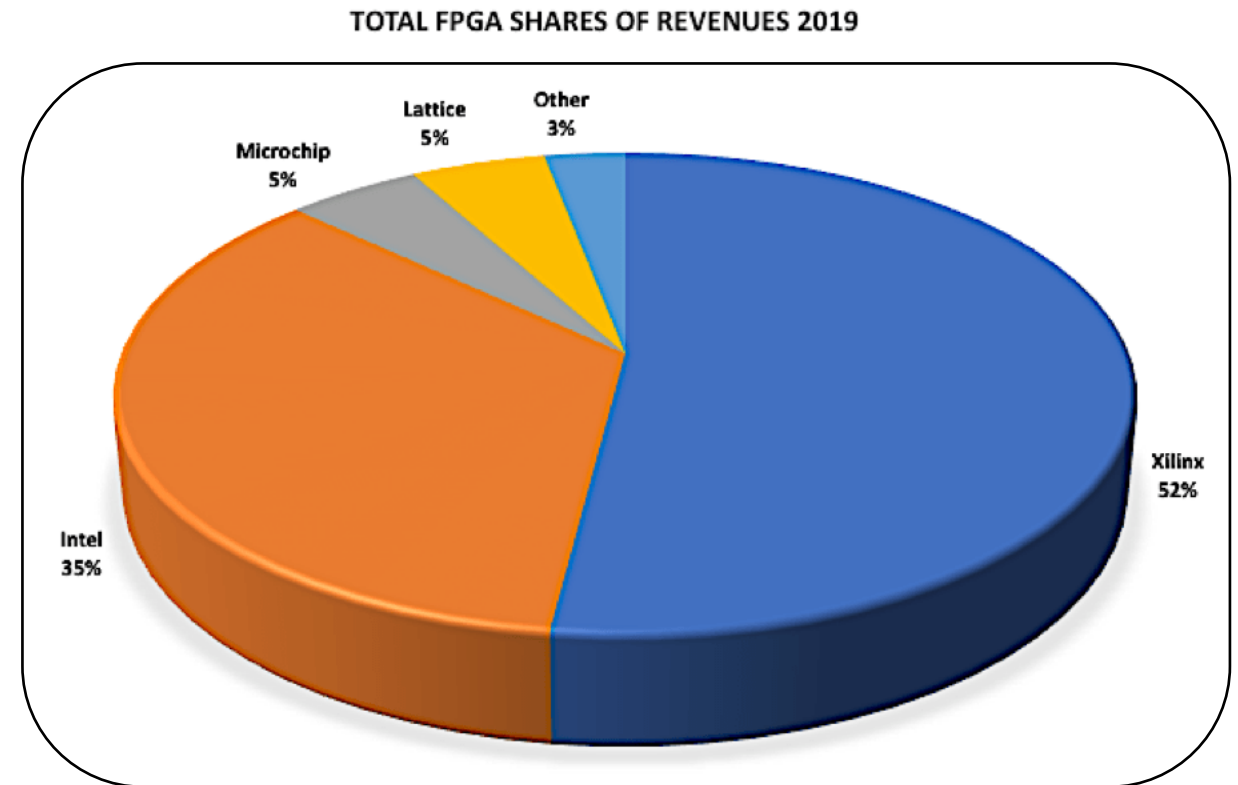
Considerando que são necessários cerca de 150 transistores para criar uma LUT4, tem-se para 50.000 LUTS (um FPGA médio pode ter tantas LUTs) → 7.500.000 transistores!!!

O MERCADO DE DISPOSITIVOS RECONFIGURÁVEIS

➤ Segmento de PLDs, 2009



➤ Segmento de FPGAs, 2019



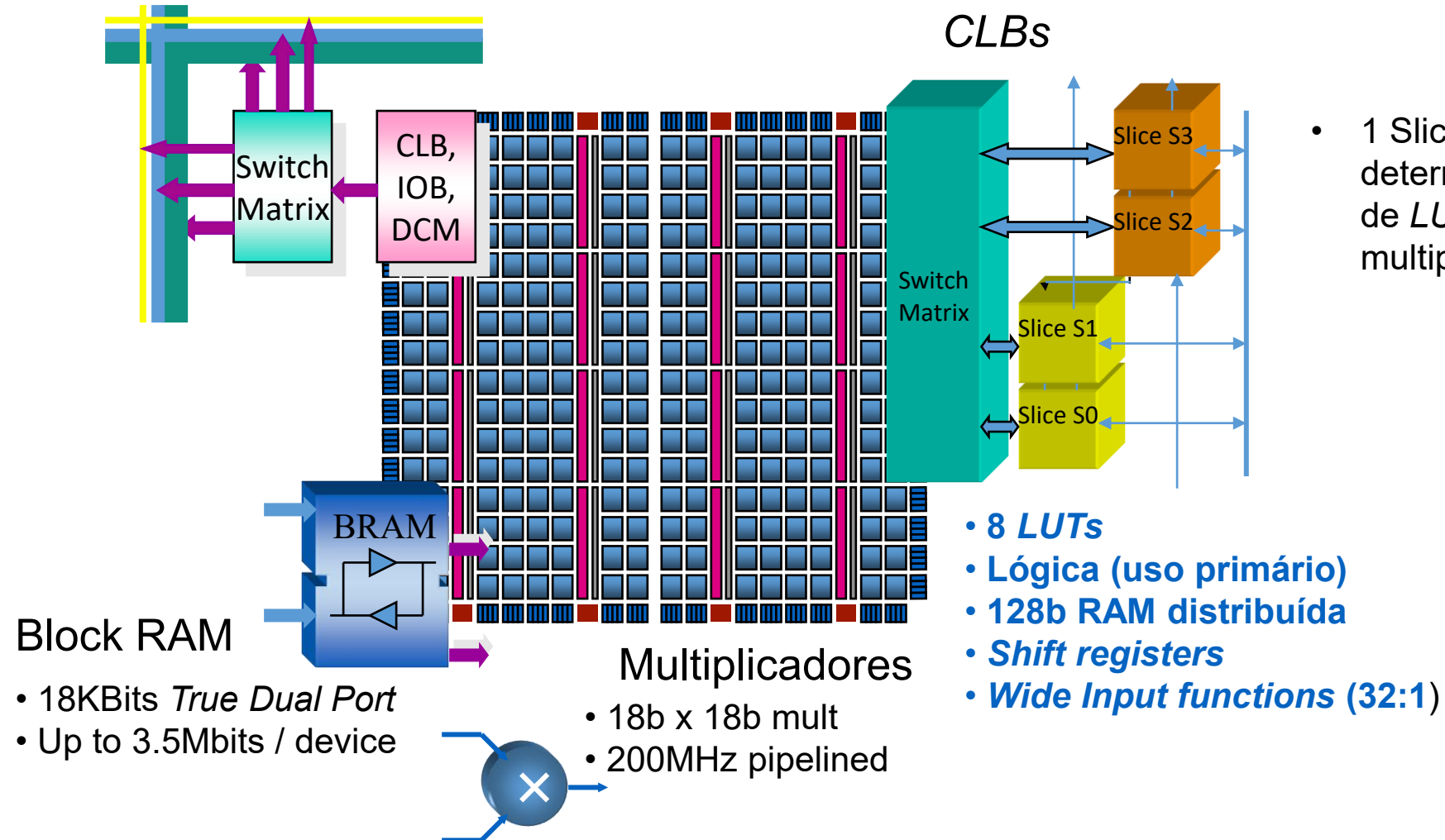
Source: The Information Network (www.theinformationnet.com)

- ✓ Apenas 2 fornecedores dominam o mercado, indicando que este está maduro

ALGUNS DISPOSITIVOS AMD E INTEL ATUAIS

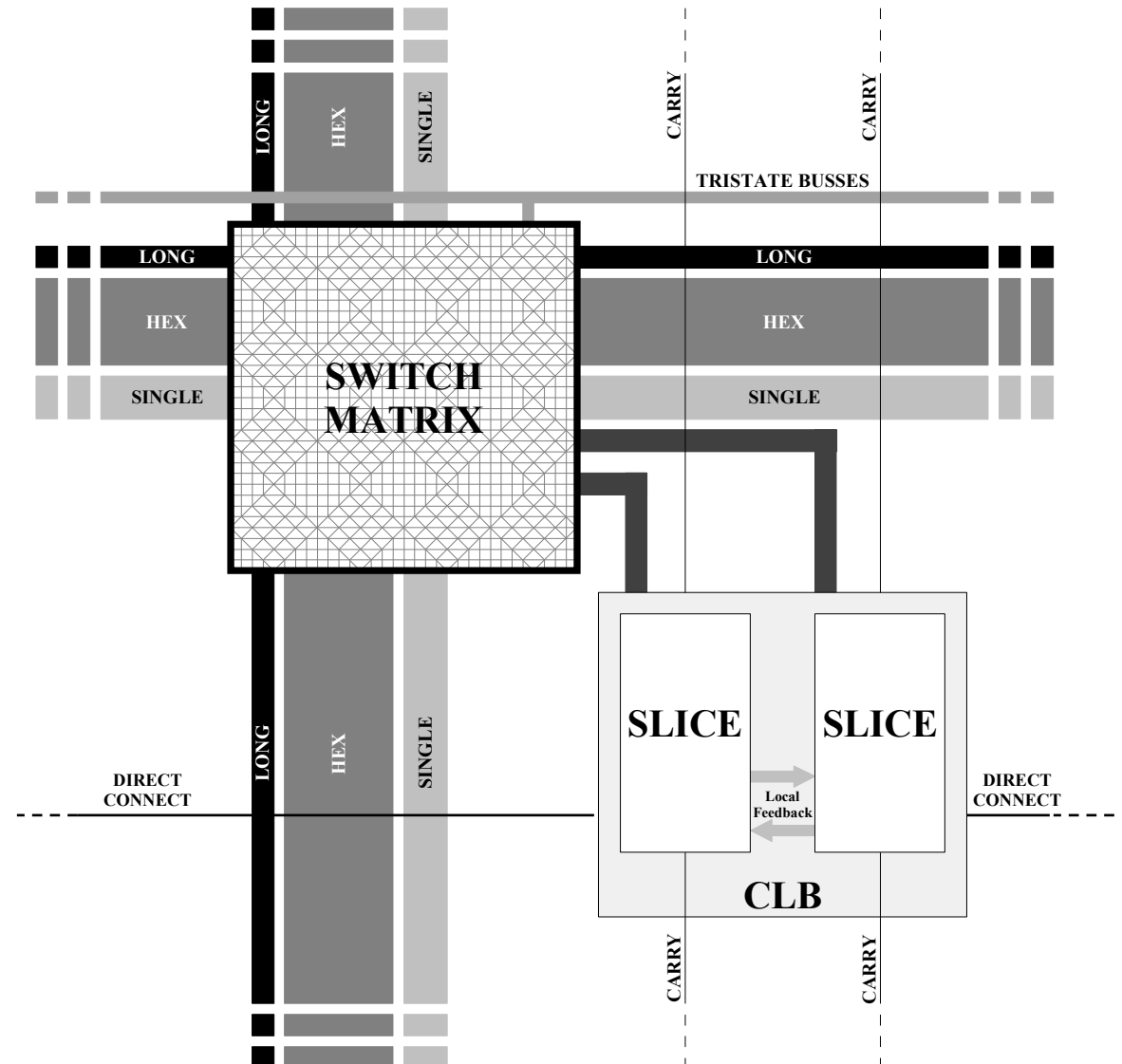
- **AMD (até 2022 era a empresa Xilinx, então adquirida pela AMD, por 60 GUS\$ em ações)**
 - ✓ Baixo/Médio Custo-Desempenho
 - Famílias Spartan 6, Spartan 7, Artix 7, Kintex 7...
 - **Spartan 3 e Spartan 3E (Temos placas, Nexys e Nexys2 com FPGA desta família)**
 - ✓ Alto desempenho
 - Virtex 7 (28nm), Virtex-Kintex UltraSCALE (20nm), Virtex-Kintex UltraSCALE+ (**16nm**), Versa (**7nm**)
- **Altera (até 2015 era Altera, foi adquirida pela Intel, por 16,7 GUS\$, hoje voltou a Altera)**
 - ✓ Baixo Custo e Mid-Range
 - Famílias Cyclone
 - **Max10 (Temos placa, MAX10, com FPGA desta família), Arria...**
 - ✓ Alto desempenho
 - Stratix, Agilex (**10nm**)...

ALGUNS DETALHES: FAMÍLIA VIRTEX II (2001-2002)



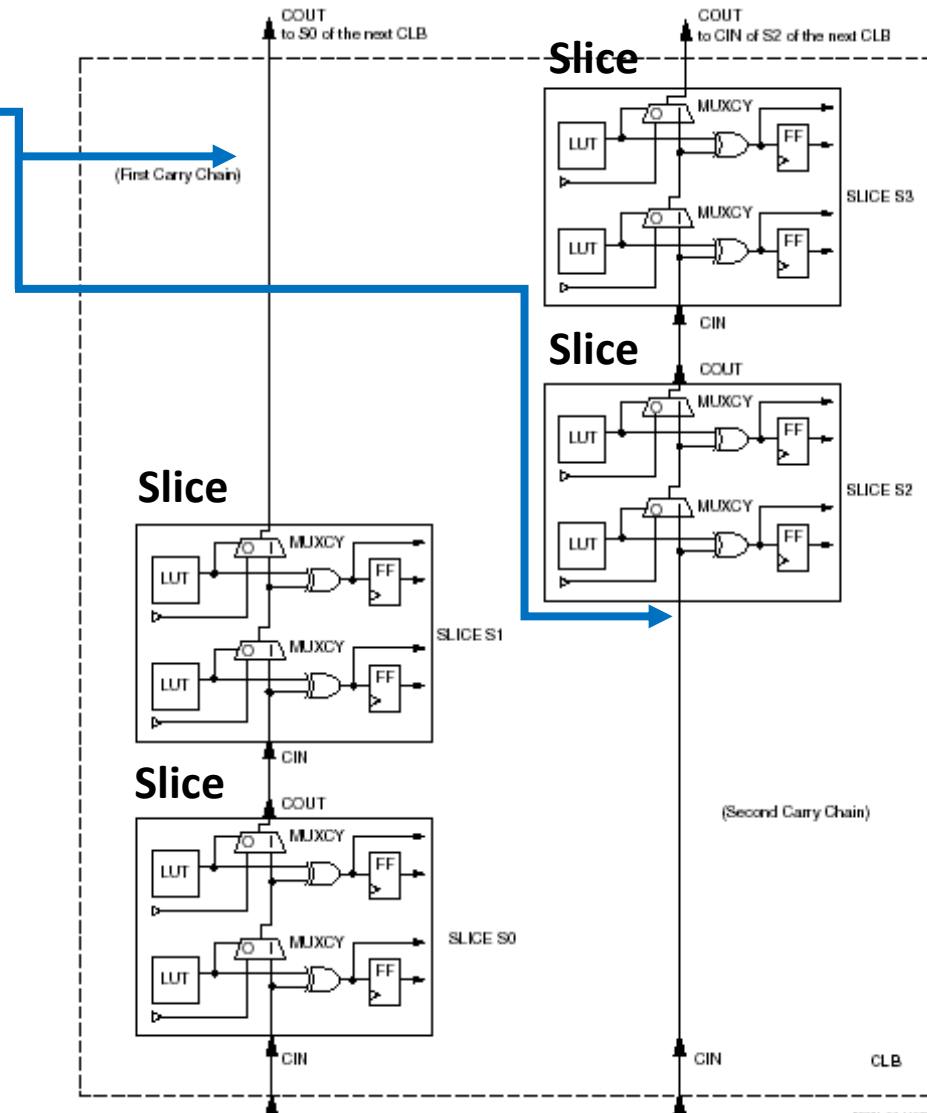
ARQUITETURA VIRTEX II – CLB E INTERCONEXÃO

- Conexões diretas entre *CLBs* vizinhas
 - ✓ Lógica de vai-um
- Matrix de conexão
 - ✓ *CLB* às linhas de roteamento
- Linhas de roteamento
 - ✓ Simples
 - ✓ Hexas
 - ✓ Longas
 - ✓ *Tri-state*

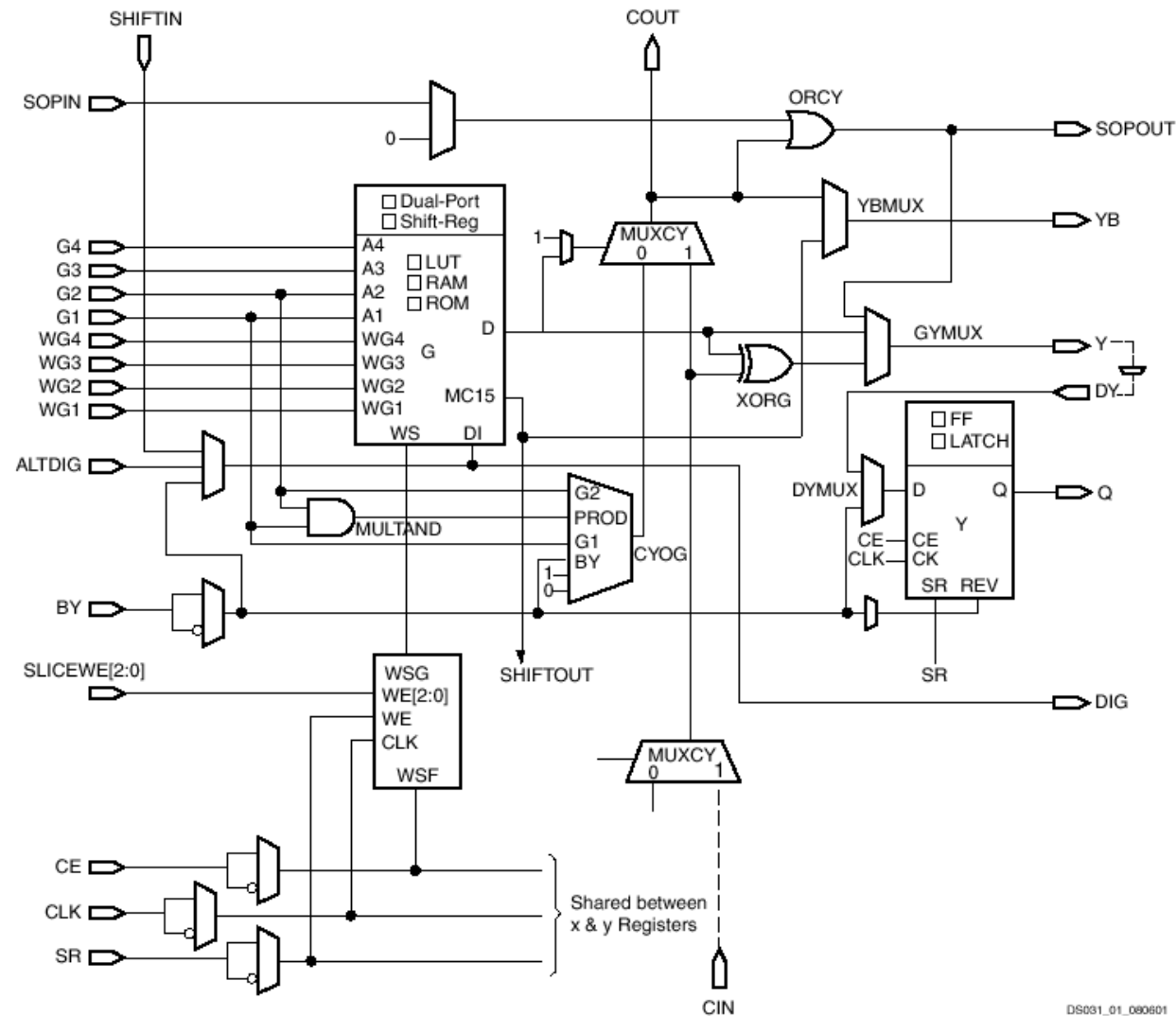


ARQUITETURA DO *CLB* DO DISPOSITIVO VIRTEx II

- *Fast Carry Logic Path*
- *Provides fast arithmetic (+,-,*,/)*
- Resumindo, um CLB na Virtex-II contém
 - ✓ 4 Slices
 - ✓ 8 LUTS / 8 Flip-Flops
 - ✓ 2 cadeias de propagação rápida de vai-um
 - ✓ 64 bits para compor uma memória (RAM/ROM)
 - ✓ 64 bits para fazer *shift registers*



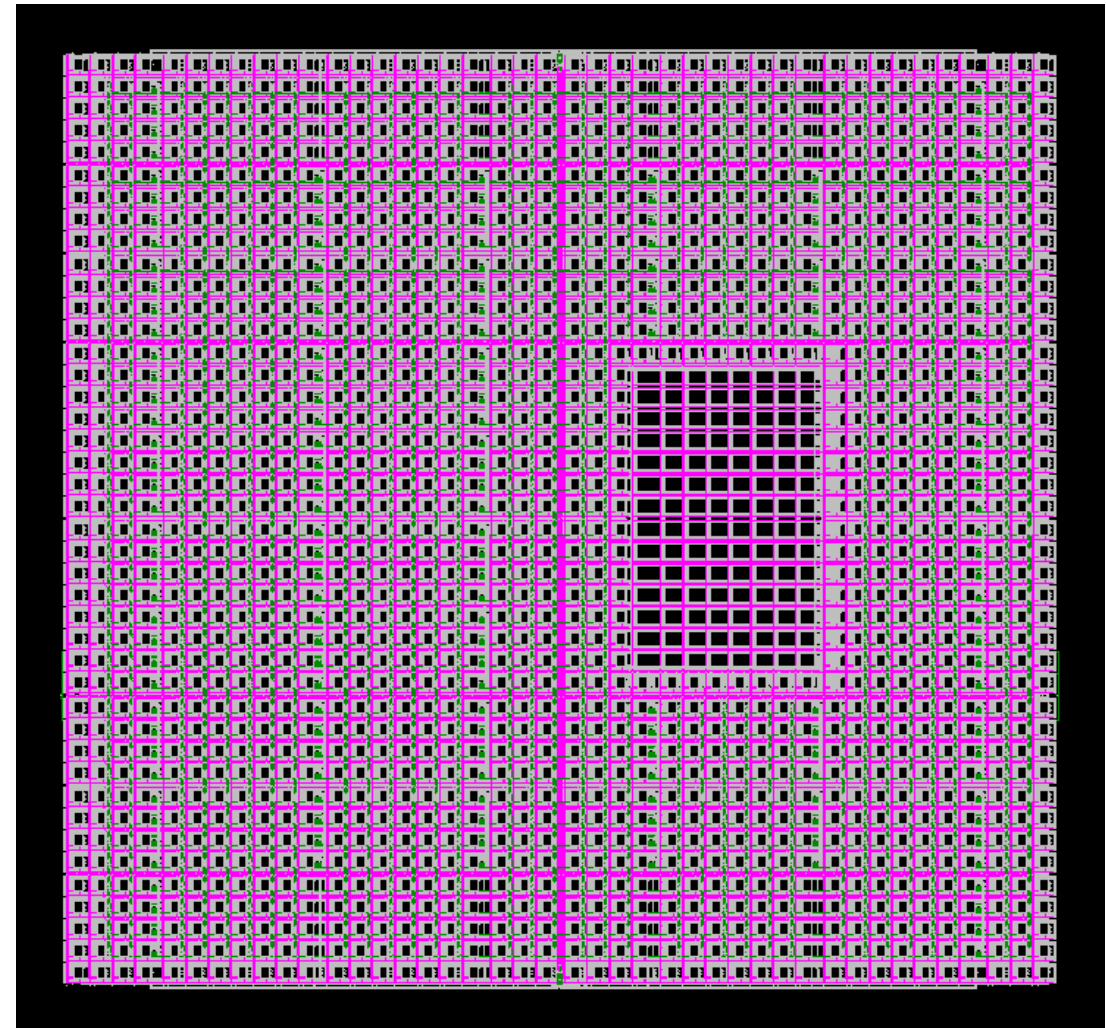
ARQUITETURA – METADE DE UMA SLICE



Um Exemplo: Na Família VIRTEX2P, o dispositivo XC2VP7

➤ FPGA Editor View With All Wires

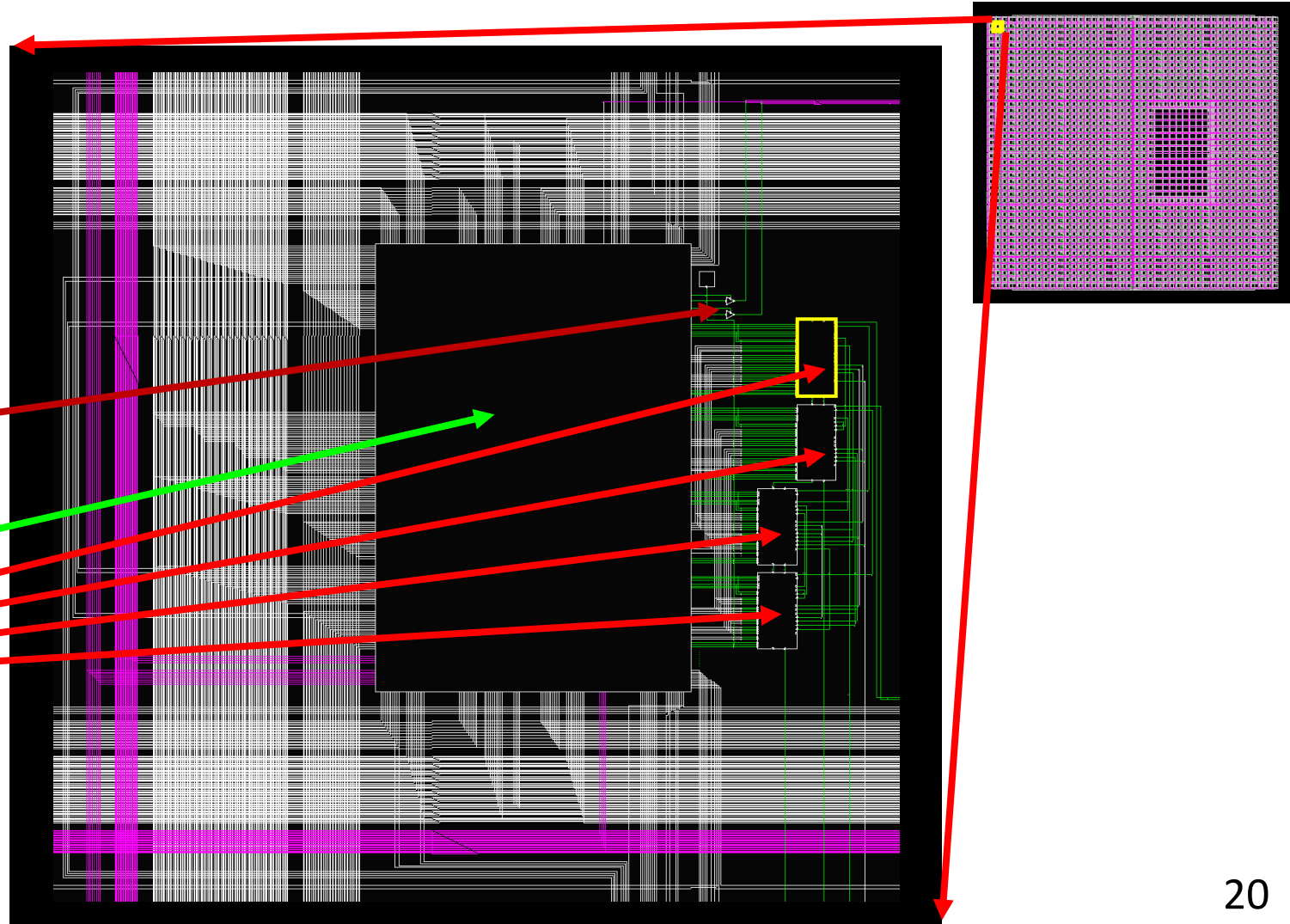
- ✓ 4,928 *slices*
- ✓ 44 *BRAMs*
- ✓ 1 PowerPC (microprocessador)
- ✓ 11,627 *logic sites*
- ✓ 2,653 *tiles*
- ✓ 1,423,681 *wires*
- ✓ 544,549 *wire segments (short)*



Um Exemplo: Na Família VIRTEX2P, o dispositivo XC2VP7

➤ FPGA Editor View With All Wires

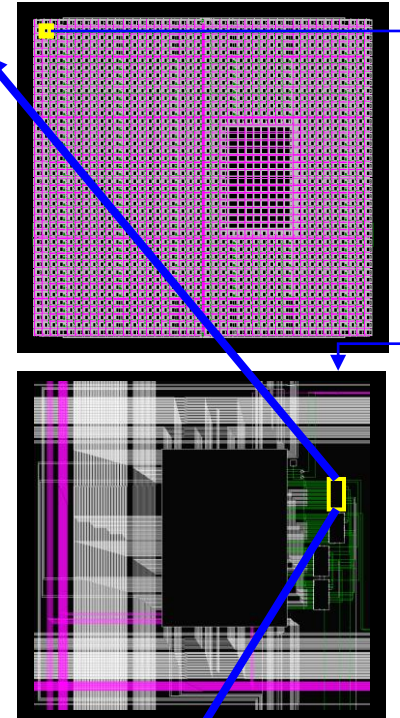
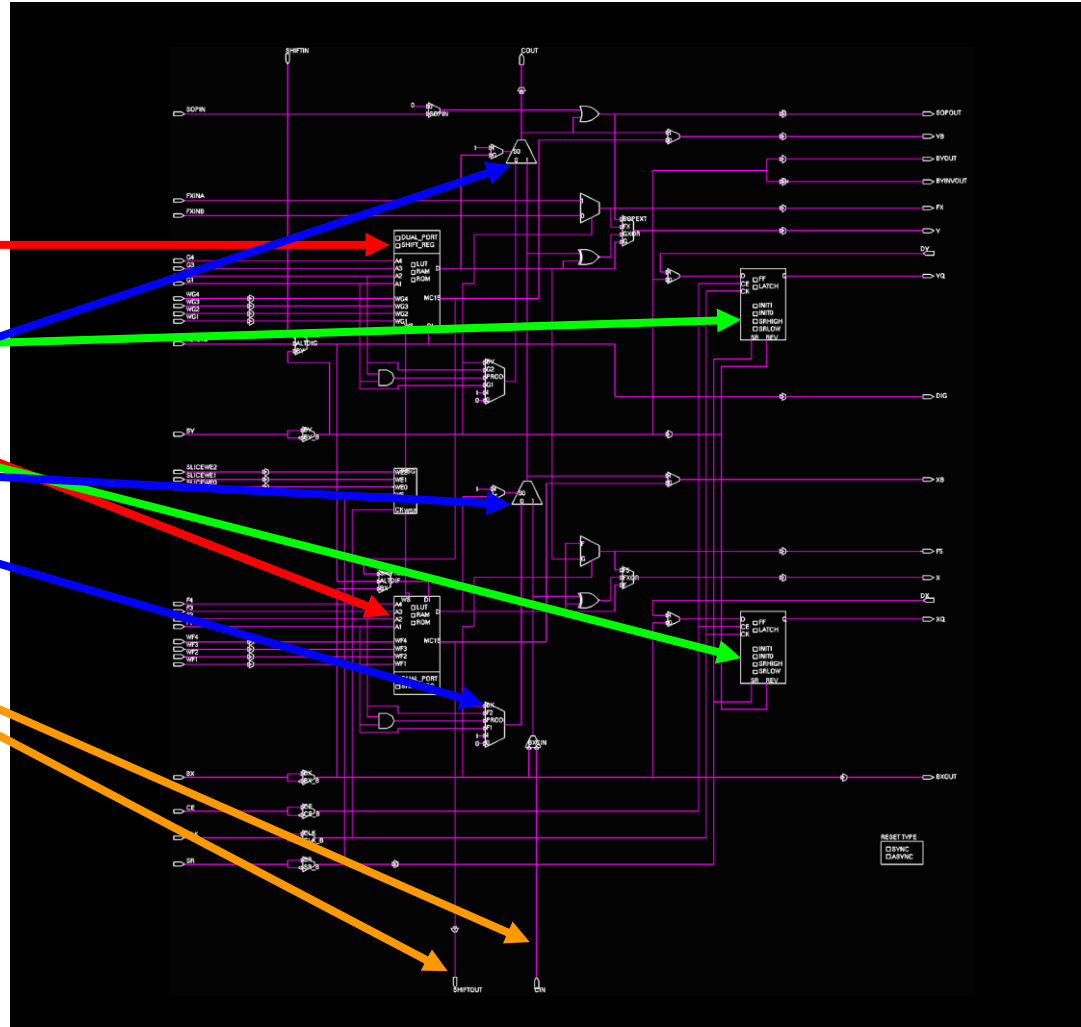
- ✓ Zoom de um *CLB* do canto superior esquerdo
- ✓ Muitos recursos de roteamento
- ✓ 2 *TBUFs*
- ✓ Grande caixa de conexões (*switch box*)
- ✓ 4 *slices*



Um Exemplo: Na Família VIRTEX2P, o dispositivo XC2VP7

➤ FPGA Editor View With All Wires

- ✓ 1 Slice da Família *Virtex2Pro*
- 2 LUTs
- 2 flip-flops
- Vários muxes
- Lógica de vai-um dedicada



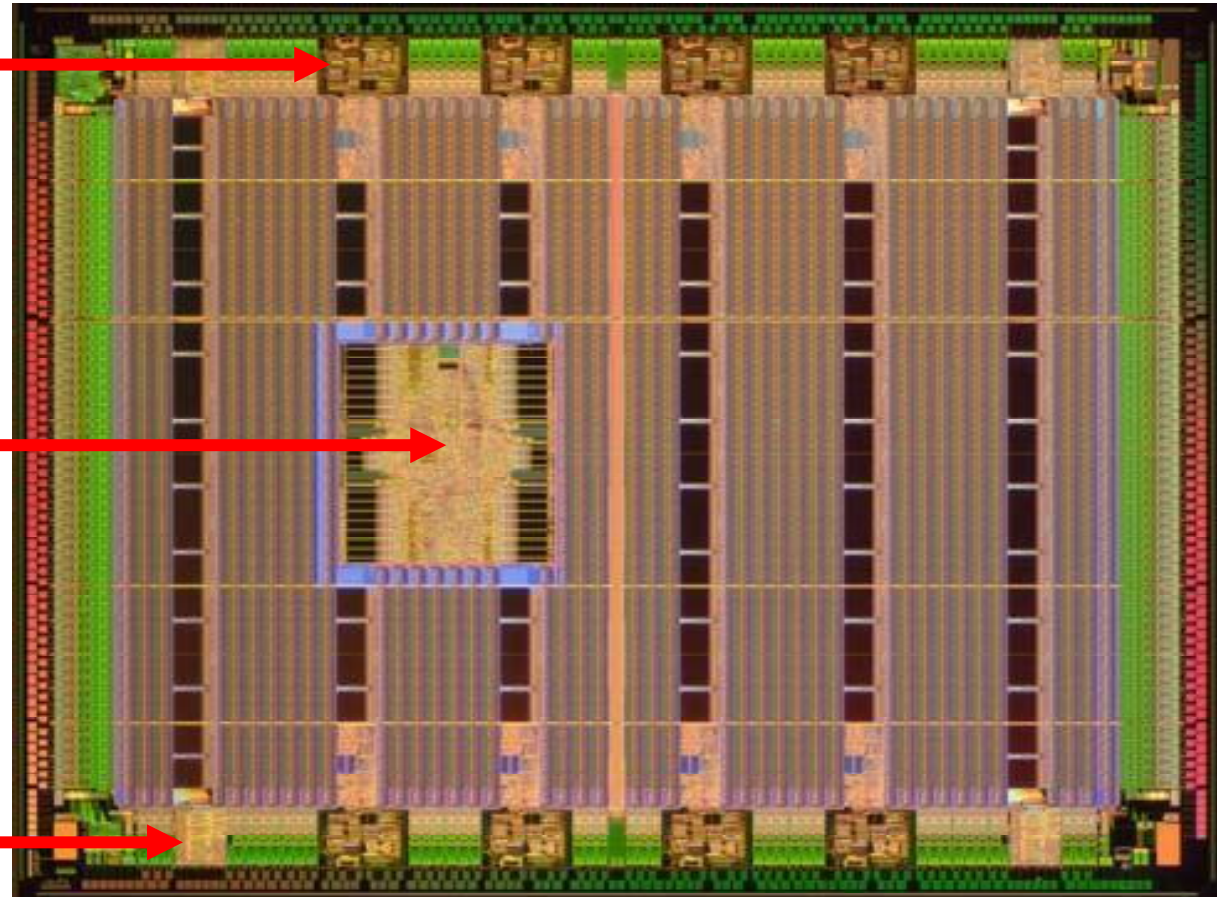
Um Exemplo: Na Família VIRTEX2P, o dispositivo XC2VP7

➤ Foto do Layout do XC2VP7

✓ MGTs (Multi-Gigabit Transceivers)

✓ Power PC

✓ DCM (Digital Clock Manager)



DEMAIS COMPONENTES DE UM FPGA MODERNO

- Vários circuitos prontos existem dentro do FPGA para o projetista usar (*Hard IPs*), e.g.
 - Para gerenciamento do(s) sinal(ais) de relógio (*clock*)
 - ✓ Reduzem escorregamento de relógio
 - ✓ Permite multiplicar, dividir, mudar a fase da(s) frequências de entrada
 - ✓ Implementações digitais (DCM – [Xilinx](#), mais baratos) e analógica (PLL – [Altera](#), mais flexíveis)
 - Blocos de memória
 - ✓ Tipicamente blocos de 9Kbits, 18kbits ou 36Kbits na Xilinx (Altera tem mais variedade de tamanhos de blocos de memória)
 - Blocos DSP
 - ✓ Multiplicadores 18bitsX18bits configuráveis, muito usados para funções de imagem, áudio, telecomunicações, IA
- Outros circuitos existem já prontos para configurar no FPGA, de graça (os mais simples), ou por um preço (os mais complexos). Estes são denominados *Soft IPs*

DEMAIS COMPONENTES DE UM FPGA MODERNO

➤ Hard IPs

- ✓ Processadores embarcados do tipo hard IP
 - ✓ Xilinx disponibiliza hoje o processador ARM em algumas famílias (e.g. Zynq)
 - ✓ Podem executar sistemas operacionais embarcados como Linux
- ✓ Criptografia do *bitstream*
- ✓ Controle para reconfiguração interna (de dentro do FPGA - ICAP)

➤ Soft IPs

- ✓ Medium Access Control (MAC) para redes Ethernet e/ou outros tipos de rede
 - ✓ MACs são protocolos da camada 2 (modelo OSI/ISO – Redes)
 - ✓ Outros tipos de MAC existem para Bluetooth, WiFi, Zigbee etc.
- ✓ Processadores Soft – e. g. MicroBlaze ([Xilinx](#)), Nios ([Altera](#)) etc.
- ✓ Módulos de criptografia – AES, RSA etc.

INTRODUÇÃO À SIMULAÇÃO DE HARDWARE DIGITAL SÍNCRONO EM VHDL

INTRODUÇÃO A UMA HDL - VHDL

- VHDL → Uma linguagem para *descrever* sistemas digitais
- Existem diversas outras: **SystemC**, **Verilog**, **System Verilog**, **Handel-C**, **ISP** etc...
- Na prática, VHDL e [System]Verilog são as mais usadas
 - Sugestão → estudem [System]Verilog também!!
- Origem
 - ✓ **VHDL**: **V**HSIC **H**ardware **D**escription **L**anguage
 - Criado no programa do Department of Defense (DoD) dos EUA: “*Very High Speed Integrated Circuits*” (VHSIC), de 1980
 - Uma das linguagem padronizadas pelo “*Institute of Electrical and Electronics Engineers*” (IEEE) em 1987, padrão revisado em 1993, 2000, 2002, 2004, 2009,...
 - ✓ Feita para *especificar* hardware → Atualmente serve também para *simulação* e *síntese*!
 - ✓ Linguagem utilizada mundialmente por empresas de Electronic Design Automation (ECAD), adotada frequentemente na Europa e no Japão
 - ✓ **Verilog** é muito usada nos EUA e junto com **SystemVerilog**, muito usada em *chip design*

HDLs - BENEFÍCIOS - VANTAGENS

➤ Especificação de um sistema digital

- ✓ Projetos ***independentes*** da tecnologia (implementação física é postergada)
- ✓ Ferramentas de *Electronic Design Automation* (EDA)/Electronic Computer Aided Design (ECAD) compatíveis entre si → EDA e ECAD são sinônimos...
- ✓ Permite explorar, em um nível mais alto de abstração (em relação e. g. a diagramas de esquemáticos), diferentes alternativas de implementação
- ✓ Permite, através de simulação, verificar o comportamento do sistema digital

➤ Nível físico

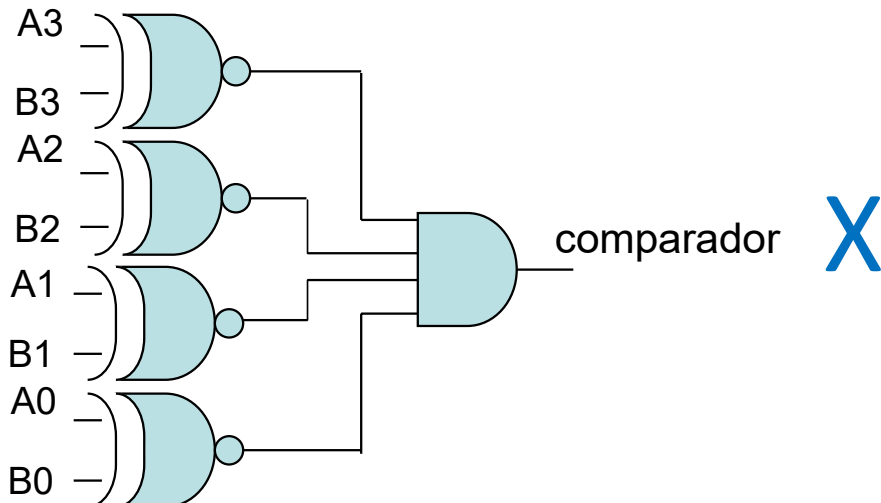
- ✓ Reduz tempo de projeto (favorece níveis abstratos de projeto)
- ✓ Reduz custo do projeto
- ✓ Elimina erros de baixo nível (se usado como base de ferramentas automatizadas)
- ✓ **Consequência:** reduz “*time-to-market*” (tempo de chegada de um produto ao mercado) 27

HDLs - DESVANTAGENS - INCONVENIENTES

➤ Em relação a diagramas de esquemáticos

- ✓ Hardware gerado pode ser menos otimizado
- ✓ Controlabilidade / Observabilidade de projeto reduzidas

➤ Circuito Lógico



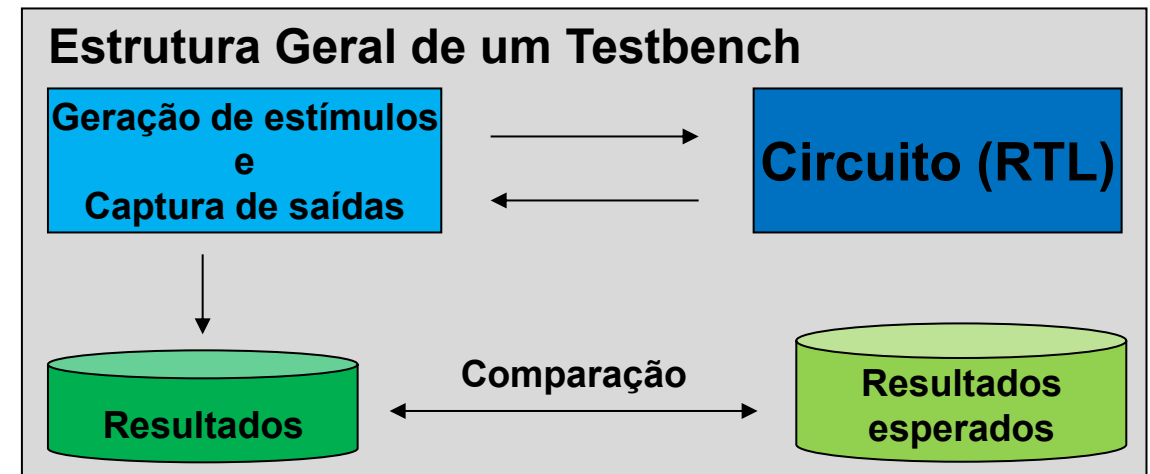
Fonte: Profa. Dra. Fernanda Gusmão de Lima
Kastensmidt. Sistemas Digitais. UFRGS.

➤ VHDL

```
signal A : in  std_logic_vector(3 downto 0);  
signal B : in  std_logic_vector(3 downto 0);  
comparador : out  std_logic;  
...  
comparador <= '1' when A=B else '0';
```

VHDL É UMA LINGUAGEM DE PROGRAMAÇÃO?

- **Enfaticamente, NÃO!!!** É uma linguagem de **descrição** de hardware!
- Código é executado em um *simulador*
 - ✓ Não se enxerga o “compilador” de VHDL, não há um “código executável” visível
→ “compilação” = “elaboração”
- Projeto do usuário
 - ✓ Especificado no nível de abstração denominado de transferência entre registradores (em inglês, *Register – Transfer Level* ou **RTL**), mas não apenas neste nível!
- **Testbench** = descrição em VHDL (ou outra linguagem) do procedimento de teste de um circuito
 - ✓ Especificação comportamental do ambiente externo ao projeto (estímulos externos)
 - ✓ Interage com o projeto
 - ✓ Não precisa ser descrito em VHDL (o projeto VHDL pode e.g. ser validado em ambiente C/C++!!)



O TIPO DE DADOS PADRÃO PARA SÍNTESE

➤ standard_ulogic

- ✓ Tipo não “resolvido”
- ✓ Enumeração com 9 níveis lógicos

```
-- Fonte: stdlogic.vhd
-----
-- logic state system  (unresolved)
-----
TYPE std_ulogic IS ( 'U',  -- Un-initialized
                    'X',  -- Forcing  Unknown
                    '0',  -- Forcing  0
                    '1',  -- Forcing  1
                    'Z',  -- High Impedance
                    'W',  -- Weak      Unknown
                    'L',  -- Weak      0
                    'H',  -- Weak      1
                    '-'   -- Don't care
                );
```

TIPO PADRÃO PARA SÍNTESE

➤ standard_logic

- ✓ Tipo “resolvido” - permite por exemplo implementação de barramentos
- ✓ Tipo mais utilizado em descrições de hardware – Vejam abaixo como ele é definido formalmente, em VHDL

```
SUBTYPE std_logic IS resolved std_ulogic;
TYPE std_logic_vector IS ARRAY ( NATURAL RANGE <>) OF std_logic;
-----
-- resolution function
-----
CONSTANT resolution_table : stdlogic_table := (
--
-- | U   X   0   1   Z   W   L   H   -   |
--
-- |-----|
-- | ('U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U'), -- | U |
-- | ('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), -- | X |
-- | ('U', 'X', '0', 'X', '0', '0', '0', '0', '0', 'X'), -- | 0 |
-- | ('U', 'X', 'X', '1', '1', '1', '1', '1', '1', 'X'), -- | 1 |
-- | ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', 'X'), -- | Z |
-- | ('U', 'X', '0', '1', 'W', 'W', 'W', 'W', 'X'), -- | W |
-- | ('U', 'X', '0', '1', 'L', 'W', 'L', 'W', 'X'), -- | L |
-- | ('U', 'X', '0', '1', 'H', 'W', 'W', 'H', 'X'), -- | H |
-- | ('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), -- | - |
-- );
```

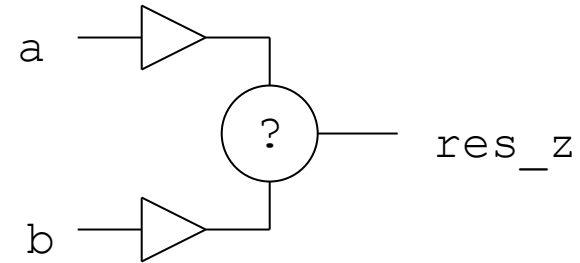
TIPO PADRÃO PARA SÍNTESE

```
signal a, b, z :<tipo pré-definido, não “resolvido”>;  
signal res_z : <tipo_resolvido>;
```

```
z <= a;  
z <= b;
```

X

- ✓ Erro de “*multiple drivers*”,
ou seja, curto-circuito

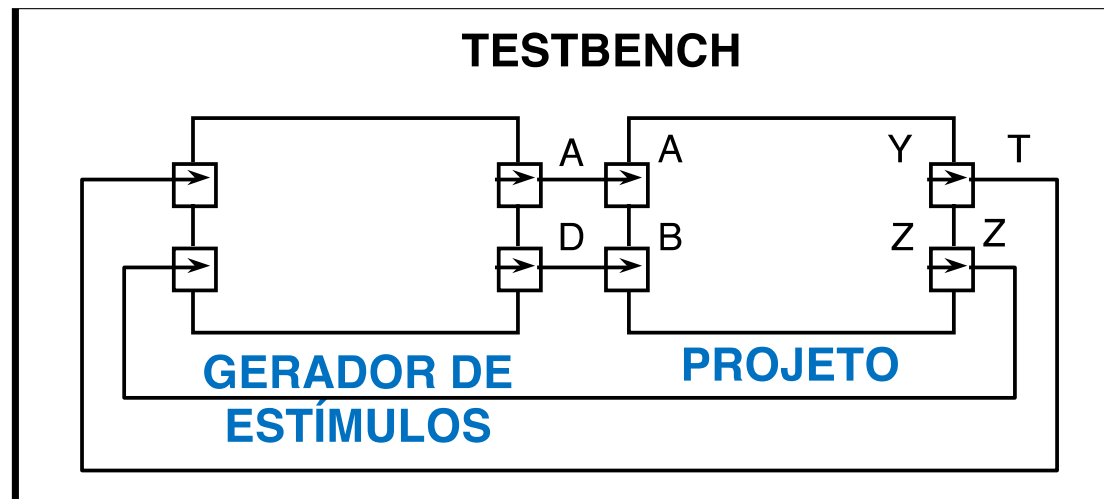


```
res_z <= a;  
res_z <= b;
```

- ✓ Resolução por tabela pré-definida
na linguagem ou em biblioteca da
linguagem (como um *header file*
em C ou Java)

VALIDAÇÃO POR SIMULAÇÃO

- **Testbenches** - Uma forma simples de testar o projeto
 - ✓ Na sua forma mais simples, um *testbench* consiste em um ou mais “processos geradores de estímulos” e uma instância do projeto que se quer testar
 - ✓ O *testbench pode ser* construído como um módulo VHDL que não contém portas de entrada/saída. Ou seja, trata-se de um sistema fechado, ou autônomo!



COMO DESCREVER HARDWARE

➤ Entity – Interface Externa

- ✓ Especifica somente a interface entre o hardware e o ambiente
- ✓ Não contém definição do comportamento ou da estrutura internos do Hw

Código VHDL da entidade

```
entity halfadd is
port (A, B: in std_logic;
      Sum, Carry: out std_logic);
end halfadd;
```

Esquemático da Interface

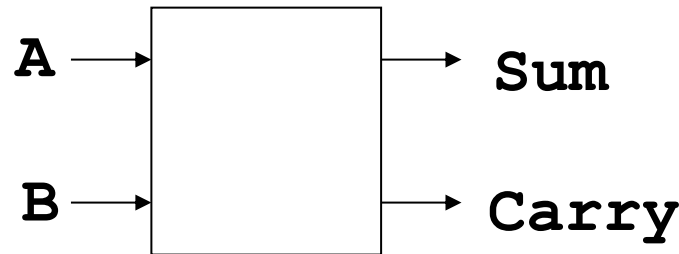


Tabela Verdade

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

COMO DESCREVER HARDWARE

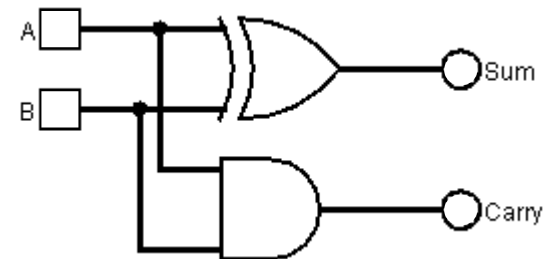
➤ **Architecture** – Comportamento do Hw

- ✓ Especifica o comportamento e/ou a estrutura internos de uma *entity*
- ✓ Deve ser associada a exatamente uma *entity* específica
- ✓ Uma *entity* pode ter associada a si várias *architecture* (representando diferentes formas de implementar uma mesma interface Hw)

Esquemático equivalente desta arquitetura

Código VHDL da arquitetura

```
architecture comp of halfadd is  
begin  
    Sum    <= A xor B;  
    Carry  <= A and B;  
end comp;
```



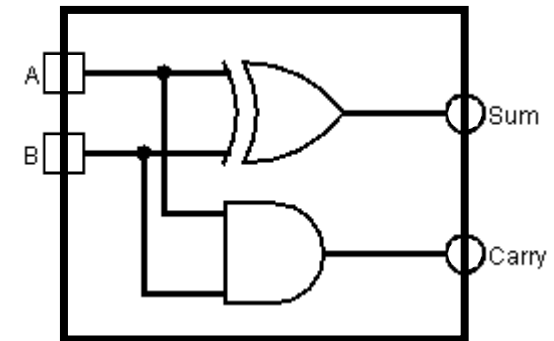
UMA DESCRIÇÃO COMPLETA DE HARDWARE

```
library IEEE;
use IEEE.std_logic_1164.all;

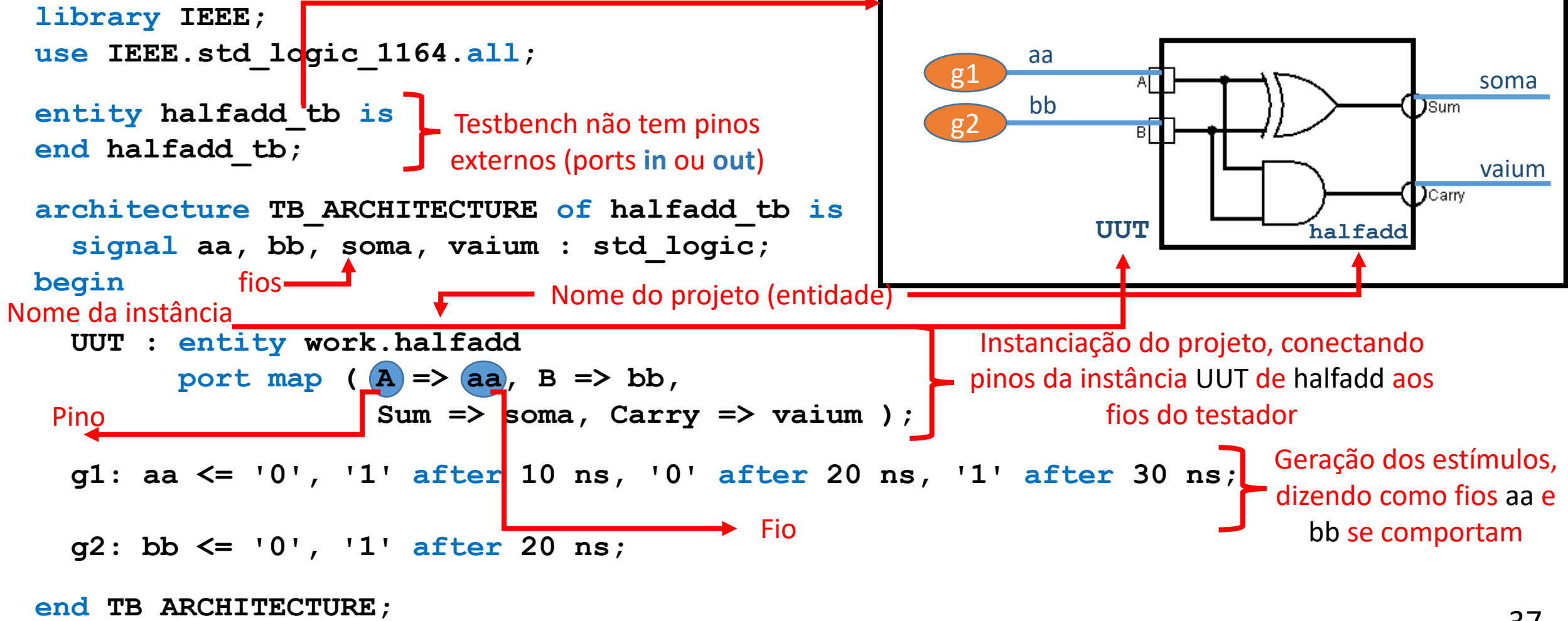
entity halfadd is
port (A, B: in std_logic;
      Sum, Carry: out std_logic);
end halfadd;

architecture comp of halfadd is
begin
    Sum    <= A xor B;
    Carry <= A and B;
end comp;
```

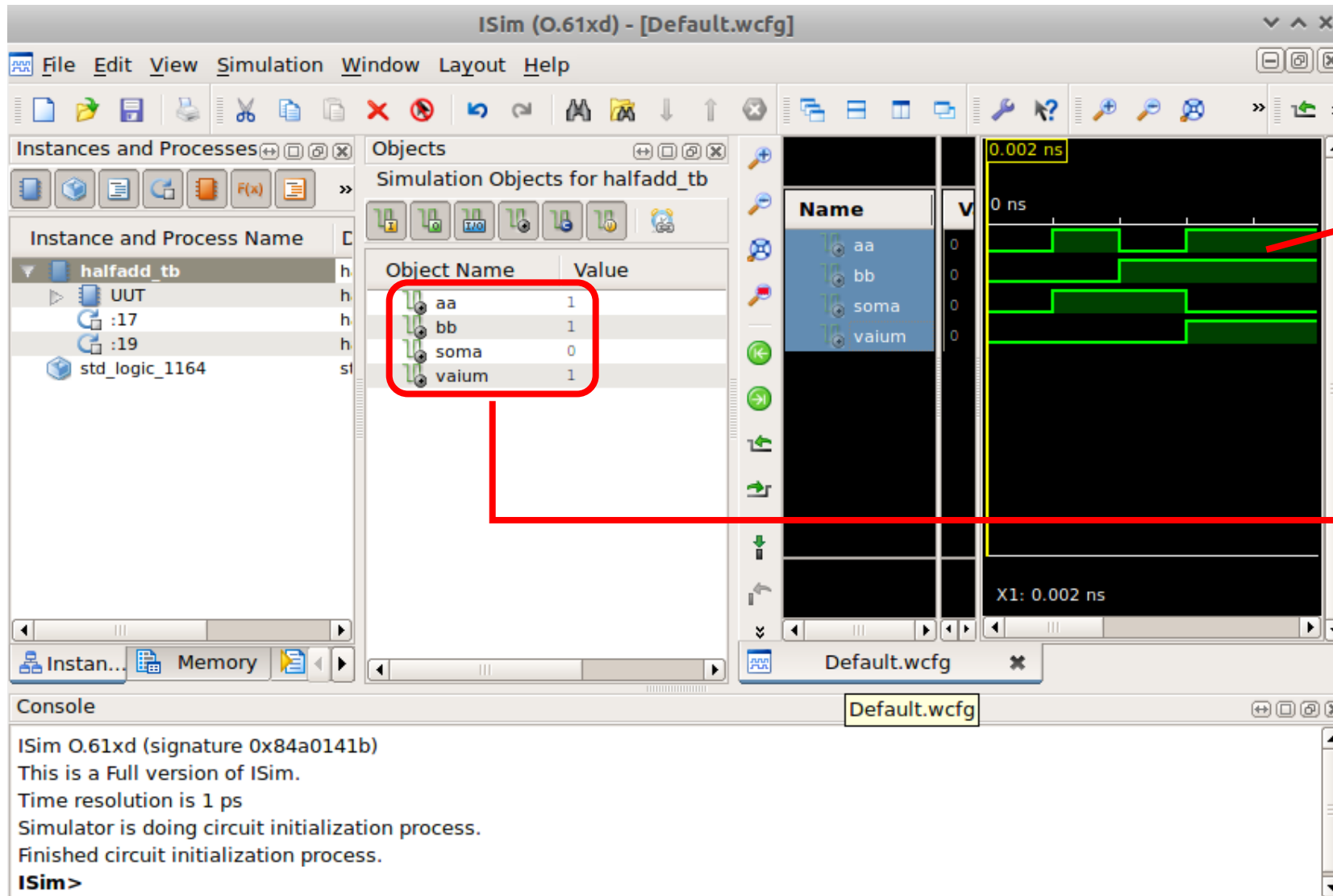
Esta biblioteca e o pacote definem o tipo `std_logic`, o qual não é parte da linguagem VHDL!



EXEMPLO DE TESTBENCH PARA UM HARDWARE



EXEMPLO DE SIMULAÇÃO DE UM HARDWARE



Notem que esta simulação de 50ns cobre todos os (4) casos da Tabela Verdade para as duas funções Booleanas implementadas! Confirmam como os casos foram definidos nas linhas com rótulos g1 e g2 do Testbench...

Sinais do Testbench:

aa – fio ligado no pino A

bb – fio ligado no pino B

soma – fio ligado no pino Sum

vaum – fio ligado no pino Carry